

Arbre de decision

Indice Max Minority

Le choix du meilleur découpage (split) d'un noeud se fait via l'indice de Gini ou l'Entropie.

L'**objectif** de cette métrique est de mesurer la pureté d'un nœud en évaluant sa capacité à isoler ou à écraser la classe minoritaire de manière linéaire.

Pour un nœud t contenant N individus, la pureté $P(t)$ est définie par la proportion de la classe majoritaire :

$$P(t) = \max_{c \in \{0,1\}} (n_c / N)$$

Exemple : Si un nœud contient 90% de feuilles saines et 10% de feuilles malades, sa pureté est $P(t) = \max(0.9, 0.1) = 0.9$. Si le nœud est parfaitement mélangé (50/50), alors $P(t) = 0.5$

Algorithme de calcul du meilleur split

Pour une variable continue, on doit implémenter la recherche du seuil optimal par balayage:

- Trié par ordre croissant les valeurs de la variables
- Pour chaque seuil candidat s (calculé comme le milieu entre deux valeurs consécutives uniques), séparez virtuellement les données en deux sous-groupes : Gauches (G) et Droites (D).

ex: on possède une liste de valeurs triées: 15, 20, 22, 34

. les milieux ou seuils des valeurs seront:

$$(15+20)/2 = 17.5 \quad \text{et} \quad (22+34)/2 = 28$$

.On prend d'abord le premier seuil qui est 17.5 puis on stocke à gauche toutes les valeurs inférieures.

.Puis on calcul la pureté à gauche et à droite

- On calcul la **Pureté pondérée du split**

$$P_{\text{split}} = (|G|/N) * P(G) + (|D|/N) * P(D)$$

Où $|G|$ et $|D|$ représentent le nombre d'éléments à gauche et à droite, et N le nombre total d'éléments dans le nœud parent.

- On prend le seuil avec le meilleur split

#exemple de code en python

#ici on a utilisé les indices lors du trie

```
def trouver_meilleur_split(self, X_column, y):
    N = len(X_column)

    def purete(y):
        n = len(y)
        sain_parmiX = len(y[y==0])
        malade_parmiX = len(y[y==1])

        pourcentage_sain = sain_parmiX / n
        pourcentage_malade = malade_parmiX / n

        return max(pourcentage_sain, pourcentage_malade)

    X_index_sorted = X_column.sort_values(ascending=True).index
    X_sorted = X_column.loc[X_index_sorted]
    seuils = []
    for i in range(N-1):
        if X_sorted.iloc[i] != X_sorted.iloc[i+1]:
            milieu = (X_sorted.iloc[i] + X_sorted.iloc[i+1]) / 2
            seuils.append(float(milieu))

    best_split = {"valeur": -math.inf, "seuil": -1, "indice_gauche": None, "indice_droite": None}

    for milieu in seuils:
        G = X_index_sorted[X_sorted < milieu]
        D = X_index_sorted[X_sorted >= milieu]
        y_G = y.loc[G]
        y_D = y.loc[D]

        purete_G = purete(y_G)
        purete_D = purete(y_D)

        Psplit = len(G)/N * purete_G + len(D)/N * purete_D

        if best_split["valeur"] < Psplit:
            best_split["valeur"] = Psplit
            best_split["seuil"] = milieu
            best_split["indice_gauche"] = G
            best_split["indice_droite"] = D
```

```
return best_split
```

Arbre de decision Max Minority

Ceci consiste à créer une fonction recursive pour construire un arbre(ensemble de noeud) regroupant les décisions à prendre pour chaque seuil, et pouvoir faire une prédiction(classer).

Le processus ne s'arrête que lorsque le noeud est 100% pur c'est à dire $P(t) = 1$ ou si l'arbre atteint une certaine profondeur.

Voici un exemple de code résumant cela

```
import math
```

```
class Noeud:
```

```
    def __init__(self, feature=None, split=None, seuil=None, droite=None, gauche=None, prediction=None):
        self.feature = feature
        self.split = split
        self.seuil = seuil
        self.droite = droite
        self.gauche = gauche
        self.prediction = prediction
```

```
class DecisionTree:
```

```
    def __init__(self, max_depth=100):
        self.max_depth = max_depth
```

```
    def fit(self, X, y):
        self.racine = self.__build_tree(X, y, max_depth=self.max_depth)
```

```
    def __build_tree(self, X, y, depth=0, max_depth=100): # Construction de l'arbre
        if depth >= max_depth or y.nunique() == 1:
            sain_parmiX = len(y[y==0])
            malade_parmiX = len(y[y==1])
            return Noeud(prediction=0 if sain_parmiX > malade_parmiX else 1) # On considère que
```

```
split = self.__trouver_meilleur_split_dataset(X, y)
```

```
if split["feature"] is None:
    sain_parmiX = len(y[y==0])
    malade_parmiX = len(y[y==1])
    return Noeud(prediction=0 if sain_parmiX > malade_parmiX else 1)
```

```
X_gauche = X.loc[split.get("indice_gauche")]
y_gauche = y.loc[split.get("indice_gauche")]
```

```
X_droite = X.loc[split.get("indice_droite")]
y_droite = y.loc[split.get("indice_droite")]
```

```
gauche = self.__build_tree(X_gauche, y_gauche, depth+1, max_depth)
droite = self.__build_tree(X_droite, y_droite, depth+1, max_depth)
```

```
return Noeud(
    feature=split["feature"],
    split=split["split"],
    seuil=split["seuil"],
```

```

        droite=droite,
        gauche=gauche,
        prediction=None
    )

def predict(self, X):
    predictions = []

    for _, ligne in X.iterrows():
        predictions.append(self.__predict_one(ligne))

    sain_parmiX = predictions.count(0)
    malade_parmiX = predictions.count(1)

    return 0 if sain_parmiX > malade_parmiX else 1

def __predict_one(self, x): # Pour faire une prediction d'une ligne du dataset en parcourant
    noeud = self.racine

    while noeud.prediction is None:

        col = noeud.feature

        if x[col] < noeud.seuil:
            noeud = noeud.gauche
        else:
            noeud = noeud.droite

    return noeud.prediction

def __trouver_meilleur_split(self, X_column, y):
    N = len(X_column)

    def purete(y):
        n = len(y)
        sain_parmiX = len(y[y==0])
        malade_parmiX = len(y[y==1])

        pourcentage_sain = sain_parmiX / n
        pourcentage_malade = malade_parmiX / n

```

```

        return max(pourcentage_sain, pourcentage_malade)

X_index_sorted = X_column.sort_values(ascending=True).index
X_sorted = X_column.loc[X_index_sorted]
seuils = []
for i in range(N-1):
    if X_sorted.iloc[i] != X_sorted.iloc[i+1]:
        milieu = (X_sorted.iloc[i] + X_sorted.iloc[i+1]) / 2
        seuils.append(float(milieu))

best_split = {"valeur": -math.inf, "seuil": -1, "indice_gauche": None, "indice_droite": None}

for milieu in seuils:
    G = X_index_sorted[X_sorted < milieu]
    D = X_index_sorted[X_sorted >= milieu]
    y_G = y.loc[G]
    y_D = y.loc[D]

    purete_G = purete(y_G)
    purete_D = purete(y_D)

    Psplit = len(G)/N * purete_G + len(D)/N * purete_D

    if best_split["valeur"] < Psplit:
        best_split["valeur"] = Psplit
        best_split["seuil"] = milieu
        best_split["indice_gauche"] = G
        best_split["indice_droite"] = D

return best_split

def __trouver_meilleur_split_dataset(self, X, y): # Pour trouver le meilleur split du dataset
    columns = X.columns
    best_split = {"feature": None, "split": -math.inf, "seuil": -1, "indice_gauche": None, "indice_droite": None}
    for col in columns:
        split = self.__trouver_meilleur_split(X[col], y)
        if best_split["split"] <= split["valeur"]:
            best_split = {"feature": col, "split": split["valeur"], "seuil": split["seuil"], "indice_gauche": split["indice_gauche"], "indice_droite": split["indice_droite"]}

    return best_split

```